Set Reinforcement Learning: Principles and Approaches

Jubayer Ibn Hamid, Ifdita Hasan Orney, Chelsea Finn, Dorsa Sadigh, Noah Goodman

Stanford University

Abstract

In this note, we discuss the set reinforcement learning framework, various objectives that can be optimized in this setting, and develop apparatus for theoretical analysis. We also discuss potential future works and applications of the framework.

Contents

1 Introduction **2**

2 Set Reinforcement Learning Framework **2**
2.1 Idealized Formulation 3
2.2 Practical Formulation 4
2.3 General Algorithm 5

3 Analysis **7**

4 Polychromic Objectives **9**

5 Algorithms for Set RL **12**
5.1 Poly-EPO 12

1 Introduction

Reinforcement learning (RL) is a highly general framework in which an agent learns from its own experience interacting with an environment. Over the years, a wide range of algorithms have been developed and applied to train increasingly powerful systems. Part of the strength and flexibility of this framework lies in the diversity of available algorithms, which make it possible to accommodate a variety of learning settings. These include differences in how frequently an agent interacts with the environment, whether the agent learns a value function or a dynamics model, the extent of privileged information available during training, and so on. Arguably, the more important source of the generality comes from our ability to define the environment itself and our ability to design the reward function that agent must optimize, allowing us to use this framework for training an agent towards a wide range of capabilities. Typically, the reward function is defined over a single action that the agent took from a state and, as such, the reinforcement learning framework attempts to train the agent to take the optimal action at any state. However, there are often settings where we might want to train the agent to sample the optimal *set* of actions at any state.

Set reinforcement learning (set RL) is a framework for optimizing policies over objectives defined on sets of trajectories. The framework was introduced by [HOX+26] and, at a high level, the framework aims to learn a policy that generates an optimal set of trajectories under a specified objective. Set reinforcement learning provides freedom over what the objective function is, similar to how reinforcement learning allows us to choose the reward function. For example, [HOX+26] used the framework to study how to explicitly train the policy to balance exploration and exploitation in a set of trajectories. One can also train an agent to optimally use test-time compute, which typically first samples a set of generations before doing verification, search, or other techniques.

In this note, we will discuss the motivations and key features of the framework and analyse various objectives. Along the way, we will extend our toolkit for analysing the characteristics of algorithms in set RL. The applications we look at in this note are mostly motivated by language model (LM) post-training. In particular, most of the objectives we will investigate are centred around training an agent to search and explore during its interactions with the environment. We will also look at adjacent works. In particular, there has been a growing interest in the field of pass@k optimization, which can be viewed as a specific instantiation of set reinforcement learning, and we will discuss how, often, the specific implementation choices used here go, in some sense, against the principles of the more general framework.

In §2, we will discuss the framework itself and study what our learning goal is. In §4, we will look at a first application of set reinforcement learning to optimize the *polychromic objective*. This will help us ground the discussion enough to introduce some of the primary apparatus for theoretically understanding set reinforcement learning in §3. Finally, in §5, we discuss algorithms to train a policy using set reinforcement learning.

2 Set Reinforcement Learning Framework

We start by recalling the standard reinforcement learning framework. Consider a Markov decision process defined by state space $\mathcal{S}$, action space $\mathcal{A}$, transition dynamics distribution $p(s_{t+1} \mid s_t, a_t)$, reward function $r : \mathcal{S} \times \mathcal{A} \to \mathbb{R}$, initial state distribution ρ_0 and discount factor $\gamma \in (0, 1)$. In reinforcement learning, we aim to find the policy π_θ parametrized by $\theta \in \Theta$ that solves the following problem:

$$\max_\theta \mathbb{E}_{s_0 \sim \rho} \mathbb{E}_{\tau \sim \pi_\theta} \left[\sum_{t=0}^{\infty} \gamma^t r(s_t, a_t) \mid s_0 \right]. \tag{1}$$

Here, a trajectory τ refers to a tuple $(s_0, a_0, r_0, s_1, a_1, r_1, \cdots)$. Note that the notation here is overloaded; we write $\tau \sim \pi_\theta(\cdot \mid s_0)$ to mean that each action $a_t \sim \pi_\theta(\cdot \mid s_t)$ while the states are sampled from the MDP's transition dynamics distribution, i.e. $s_{t+1} \sim P(\cdot \mid s_t, a_t)$. A more careful notation would be to write $\tau \sim P^{\pi_\theta}$, but we can safely ignore this for now. Intuitively, the problem here is to find the policy that maximizes the expected returns from a trajectory τ sampled under the policy. One can think of this problem as essentially aiming to solve $\max_\theta \mathbb{E}_{s_0 \sim \rho, \tau \sim \pi_\theta(\cdot \mid s_0)} [f(\tau)]$ where $f(\tau)$ is the (discounted) sum of rewards in the trajectory τ. We will refer to this framework as *standard reinforcement learning* for clarity.

Set reinforcement learning is a framework that, at a high level, attempts to find that policy that maximizes the expected value of a *set* of trajectories sampled under the policy with respect to some objective function. We first look at the sequential formulation setting and, then study the more practical setting.

2.1 Idealized Formulation

In the idealized sequential formulation, at each visited state s, the policy samples a set of n actions independently from $\pi_\theta(\cdot \mid s)$ where $n > 1$ is a fixed integer. Given $a_{1:n} := (a_1, \ldots, a_n) \overset{\text{i.i.d.}}{\sim} \pi_\theta(\cdot \mid s)$, a set-level reward $f(s, a_{1:n})$ is assigned to the entire set. Repeating this procedure recursively at each timestep generates an n-ary rollout tree induced by our policy. At depth t, the tree contains n^t states, which we denote by $s_t^{(1)}, \ldots, s_t^{(n^t)}$. For each such state $s_t^{(i)}$, let $(a_t)_{1:n}^{(i)}$ denote the n-tuple of actions sampled from $\pi_\theta(\cdot \mid s_t^{(i)})$. The goal is to solve the following optimization problem defined using set value function:

$$\max_\theta \mathbb{E}_{s_0 \sim \rho} \mathbb{E}_{\pi_\theta} \left[\sum_{t=0}^{\infty} \sum_{i=1}^{n^t} \gamma^t f\left(s_t^{(i)}, (a_t)_{1:n}^{(i)}\right) \mid s_0 \right]. \tag{2}$$

To be clear, at each state $s_t^{(i)}$, the set of n actions are sampled independently, i.e. $a_{t,1}^{(i)}, \cdots, a_{t,n}^{(i)} \overset{i.i.d}{\sim} \pi_\theta(\cdot \mid s_t^{(i)})$. In particular, we require that f is an objective such that $\mathbb{E}_{a_{1:n} \sim \pi_\theta(\cdot \mid s)} [f(s, a_{1:n})]$ cannot be written as $\mathbb{E}_{a \sim \pi_\theta(\cdot \mid s)} [g(s, a)]$ where g is independent of θ for all policy parameters $\theta \in \Theta$. In other words, we cannot collapse the set RL problem to a standard RL one. This is an important feature in the definition of set RL.

This last point is particularly important. For example, if the objective is defined as $f(s, a_{1:n}) = \frac{1}{n} \sum_{i=1}^{n} r(s, a_i)$, then the set RL problem in eq. (2) becomes equivalent to the standard RL problem in eq. (1). However, one can consider several objectives that satisfy the requirements for set RL; for example, $f(s, a_{1:n}) = \max_{i \in \{1, \cdots, n\}} r(s, a_i)$.

To understand what set RL is attempting to optimize, it is useful to construct a notion of value functions in this framework.

Definition 2.1 (Set Value Functions)**.** *Given a policy π generating a state-visitation tree and a set objective $f : \mathcal{S} \times \mathcal{A}^n \to \mathbb{R}$, the set state value function $V_\pi^\sharp(s; f)$ and the set state-action value function $Q_\pi^\sharp(s, a_{1:n}; f)$ are defined as*

$$V_\pi^\sharp(s; f) = \mathbb{E}_\pi \left[\sum_{t=0}^{\infty} \sum_{i=1}^{n^t} \gamma^t f(s_t^{(i)}, (a_t)_{1:n}^{(i)}) \Big| s_0 = s \right], \tag{3}$$

$$Q_\pi^\sharp(s, a_{1:n}; f) = \mathbb{E}_\pi \left[\sum_{t=0}^{\infty} \sum_{i=1}^{n^t} \gamma^t f(s_t^{(i)}, (a_t)_{1:n}^{(i)}) \Big|_{(a_0)_{1:n} = a_{1:n}}^{s_0 = s,} \right] \tag{4}$$

Here, we assume that the discount factor $\gamma \in (0, \frac{1}{n})$ to ensure values remain bounded - the range is smaller since we add the expected sum of rewards from n actions stemming out of each state. Intuitively, $V_\pi^\sharp(s)$ is the expected discounted return of the entire state tree rooted at s, as illustrated in fig. 1, where the reward at each node is given by the set objective. In contrast, $Q_\pi^\sharp(s, a_{1:n})$ evaluates the expected return of the tree that begins at s with the specific action set $a_{1:n}$.

With these definitions, we first see that the set RL problem can be written as:

$$\max_\theta \mathbb{E}_{s_0 \sim \rho} \left[V_{\pi_\theta}^\sharp(s_0; f) \right].$$

In other words, in set reinforcement learning, we want to learn the policy that maximizes the expected return from a *tree* generated using our policy is maximized. In contrast, in standard reinforcement learning, we want to learn the policy such that the expected return from a *trajectory* is maximized.

The sequential formulation is conceptually useful, but it is computationally impractical in most settings because the number of sampled states grows exponentially with depth. We will now look at the significantly more practical formulation of the problem.

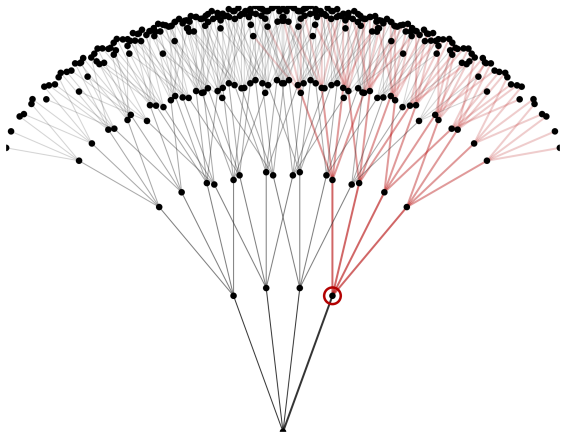

Figure 1: The set value of a state (circled) is the expected discounted return of the subtree (highlighted) rooted in this state.

2.2 Practical Formulation

In many practical settings, we do not want to sample state-visitation trees like we did in the sequential framework. For example, in the language model (LM) post-training setting, we sample a batch of prompts $x_1, \cdots, x_{|\mathcal{B}|} \sim \mathcal{D}$ where $\mathcal{D}$ is our training set, and then sample N generations from the LM conditioned on each prompt, i.e. $y_{1:n} := (y_1, \cdots, y_N) \sim \pi_\theta(\cdot \mid x_i)$ for each x_i.

In this setup, set RL aims to solve:

$$\max_\theta \mathbb{E}_{x\sim\mathcal{D}}\mathbb{E}_{y_{1:n}\sim\pi_\theta(\cdot|x)}\Big[f(x, y_{1:n})\Big]. \tag{5}$$

Again, we require that the set objective function f is such that we cannot write, for each x, $\mathbb{E}_{y_{1:n}}\Big[f(x, y_{1:n})\Big]$ as $\mathbb{E}_y\Big[g(x, y)\Big]$ where g is independent of θ for all policy parameters $\theta \in \Theta$. This requirement is crucial because, otherwise, under some objectives (for e.g., $f(x, y_1, \cdots, y_n) = \frac{1}{n}\sum_{i=1}^n r(x, y_i)$) the set RL formulation in eq. (5) becomes equivalent to a standard RL formulation using some non-differentiable reward function. Generally, if the set level objective $f(x, y_1, \cdots, y_n)$ is affine in (x, y_i) for each $i \in \{1, \cdots, n\}$, the objective does not couple the elements of the set and, as such, the framework collapses to standard RL.

We now look at some examples of objectives that do satisfy the set reinforcement learning framework.

Example 2.2. Let $f(x, y_{1:n}) = \max_i r(x, y_i)$. This objective corresponds to pass@n optimization [WK25].

Example 2.3. More generally, one can consider objectives that represent the usage of test-time compute. Let $\texttt{Alg}(x, y_1, \cdots, y_n) \in \mathcal{Y}$, i.e. the algorithm returns the final generation to return. Then, one can consider $f(x, y_{1:n}) = r(x, \texttt{Alg}(x, y_{1:n}))$ which is an objective that rewards the generation chosen by our test-time compute. Alternatively, if the algorithm returns a subset of the generations, i.e. $\texttt{Alg}(x, y_{1:n}) \in \mathcal{Y}^m$ for $m < n$, then we can set $f(x, y_{1:n}) = \mu(\{r\left(x, \texttt{Alg}(x, y_{1:n})\right)_j\}_{j=1}^m)$ which is the average reward in the set of generations selected by the algorithm.

Example 2.4. Let $d(x, y_{1:n}) \in [0, 1]$ be a measure of the diversity in the generations $(y_1, \cdots, y_n)$. Then, the following objective also satisfies set RL requirements: $f(x, y_{1:n}) = \frac{1}{n}\sum_{i=1}^n r(x, y_i) \cdot d(x, y_{1:n})$. This is the *polychromic objective*, designed to encourage the model to balance exploration and exploitation, which we will look at in detail in the next section.

To consolidate our understanding of how set RL optimizes differently from standard RL, we look at the policy gradient. After applying the score-function identity to the joint density, we get the following policy

gradient:

$$\nabla_\theta \mathbb{E}_{x\sim\mathcal{D}}\mathbb{E}_{y_{1:n}\sim\pi_\theta(\cdot|x)}\Big[f(x, y_{1:n})\Big] = \mathbb{E}_{x\sim\mathcal{D}}\mathbb{E}_{y_{1:n}\sim\pi_\theta(\cdot|x)}\left[\big(f(x, y_{1:n}) - \hat{f}(x)\big)\sum_{i=1}^{n}\nabla_\theta \log \pi_\theta(y_i \mid x)\right] \tag{6}$$

where $\hat{f}(x)$ is a set-level baseline. In other words, $\hat{f}(x)$ is independent of $y_{1:n}$ or any strict subset of $y_{1:n}$. The defining feature of (6) is that all generations in the sampled set $y_{1:n}$ share the same scalar learning signal, $f(x, y_{1:n}) - \hat{f}(x)$. In contrast, in the standard RL setting, we get:

$$\nabla_\theta \mathbb{E}_{x\sim\mathcal{D}}\mathbb{E}_{y\sim\pi_\theta(\cdot|x)}\Big[r(x, y)\Big] = \mathbb{E}_{x\sim\mathcal{D}}\mathbb{E}_{y\sim\pi_\theta(\cdot|x)}\left[\nabla_\theta \log \pi_\theta(y \mid x)\cdot\big(r(x, y) - b\big)\right]. \tag{7}$$

As such, in standard RL, each generation receives its own individual advantage. In set RL, the objective f couples the samples in the set, and the gradient assigns equal credit to all elements in the set through a shared set-level signal. For this reason, admissible baselines must be independent of the sampled set $y_{1:n}$ or any subset of it.

The notion of coupling will be a recurring theme in this note. So far, we have seen in set RL, the objectives within a set are coupled in the sense that they all receive the same score. We will soon see that, depending on the choice of the function f, multiple objectives can be “coupled", i.e. sets can be optimized to balance several goals for our learning agent.

A naive algorithm for training an agent in this set RL framework would be to sample multiple sets of size n per prompt $x \sim \mathcal{D}$ to get a Monte Carlo estimate of the gradient in eq. (6). However, there are strong reasons why the sample complexity of this naive algorithm could be harmful. In particular, if the baseline, $\hat{f}(x)$ is also a Monte Carlo estimate, one would need to sample several sets of size n for there to be a meaningful reduction in variance. We will see more practical algorithms that circumvent such issues soon.

2.3 General Algorithm

We now discuss a general recipe for optimizing a model using set reinforcement learning. This will be strongly motivated by the language model (LM) post-training setting.

Suppose, that our set objective, $f(x, y_1, \cdots, y_n)$, is defined over sets of size n. If we wanted to use an empirical estimate (e.g., a Monte Carlo estimate) of the gradient in (6), then we would need to sample several sets of size n conditioned on a single prompt which can be computationally expensive. In this section, we consider how we can instantiate set reinforcement learning using a computationally feasible gradient estimator.

For a fixed prompt x, we first draw N i.i.d. generations $y_1, \ldots, y_N \sim \pi_\theta(\cdot \mid x)$. We require that the number of samples, N, is greater than the size of sets, n, in the set RL framework. Next, from these samples, we construct K sets, $G_1, \cdots, G_K$, of size $n < N$, in a combinatorial fashion without replacement:

$$G_j = \{y_{(j_1)}, \ldots, y_{(j_n)}\}, \quad j_1 < \cdots < j_n, \{j_1, \ldots, j_n\} \subseteq \{1, \ldots, N\}.$$

We set K to be equal to $\binom{N}{n}$, which is the maximum number of sets we can construct in this manner, since this allows our final gradient to be an unbiased estimator of eq. (6). Now, given our set RL objective function f, each set G_j is assigned a set-level score

$$f(x, G_j) = f\big(x, y_{(j_1)}, \ldots, y_{(j_n)}\big).$$

Note that this allows us to bypass the sample complexity issue that we had encountered in our naive set RL algorithm, where we sampled sets of size n independently. In particular, we get $\binom{N}{n}$ sets of size n by sampling only N generations. However, if scoring each set (i.e. computing $f(x, y_{1:n})$) is computationally expensive, then the algorithm could still become computationally impractical. This is an important consideration for us as we think about what objectives we want to optimize and how we would implement them.

Algorithm 1 On-Policy Implementation of Set RL

Require: Policy π_θ, batch of inputs B, number of rollouts N, set size (for set RL) n

for each input $x \in B$ **do**
  Sample $y_1, \ldots, y_N \sim \pi_\theta(\cdot \mid x)$
  // Construct sets either by enumerating all $\binom{N}{n}$ subsets or by uniformly sampling K subsets from them
  Construct sets $G_1, \cdots, G_K$ from $\{y_{1:N}\}$ without replacement
  Compute score of each set $\{f(x, G_l)\}_{l=1}^K$ and baseline $b = \frac{1}{K}\sum_{l=1}^K f(x, G_l)$
  Compute set advantage of each set $A^\sharp(x, G_l) = f(x, G_l) - b$ for $l = 1, \cdots, K$
  Compute marginal set advantage $\widehat{A^\sharp_{\text{marg}}}(x, y_j) = \frac{1}{p}\sum_{m=1}^p A^\sharp(x, \mathcal{G}(y_j)_m)$ for $j = 1, \cdots, N$
  $\hat{g}(x) \leftarrow \sum_{j=1}^N \nabla_\theta \log \pi_\theta(y_j \mid x) \cdot \widehat{A^\sharp_{\text{marg}}}(x, y_j)$
end for
$\hat{g} \leftarrow \frac{1}{|B|}\sum_{x \in B} \hat{g}(x)$
return $\hat{g}$

Set advantage. We construct a variance-reduction baseline using the sampled sets:

$$\hat{f}(x) := \frac{1}{K}\sum_{j=1}^K f(x, G_j).$$

In other words, the baseline is simply an average of the scores of all sets under our objective function f. Note that since we use a Monte Carlo estimate of the baseline, it is useful to construct as many sets as possible so that the variance in the baseline estimate is decreased. Given the baseline, for each set G_j, we can now construct a set advantage function which is simply

$$A^\sharp_{\pi_\theta}(x, G_j; f) = f(x, G_j) - \hat{f}(x). \tag{8}$$

Marginal set advantage of a single generation. We now derive an advantage function at the level of a single generation. Let $\mathcal{G}(y_i)_1, \ldots, \mathcal{G}(y_i)_p$ denote all sets that contain y_i, where p is the number of sets in which each generation appears under the chosen construction. We define the marginal set advantage of a single generation:

$$\widehat{A^\sharp_{\text{marg}}}(x, y_i) := \frac{1}{p}\sum_{j=1}^p A^\sharp_{\pi_\theta}(x, \mathcal{G}(y_i)_j; f). \tag{9}$$

Final Estimator. Using this, we use the following set RL gradient estimator:

$$\nabla_\theta \widehat{\mathbb{E}}_{x \sim \mathcal{D}, y_{1:n} \sim \pi_\theta(\cdot|x)}\left[f(x, y_{1:n})\right] = \widehat{\mathbb{E}}_{x \sim \mathcal{D}, y_1, \cdots, y_N \sim \pi_\theta(\cdot|x)}\left[\sum_{i=1}^N \nabla_\theta \log \pi_\theta(y_i \mid x) \cdot \widehat{A^\sharp_{\text{marg}}}(x, y_i)\right] \tag{10}$$

Now, we have all the ingredients required for optimizing an LM with respect to the set RL framework. Since the marginal set advantage allows gives us an advantage function over single generations, we can use existing *standard* RL algorithms, such as `PPO` or `GRPO`, to optimize the set RL objective by replacing their standard advantage, $A(x, y_i)$, with $\widehat{A^\sharp_{\text{marg}}}(x, y_i)$. The pseudocode for this general recipe is provided in algorithm 1.

We can easily show that our algorithm uses an unbiased estimator of eq. (6) up to a scaling of the learning rate as follows:

Proposition 2.5. *Fix a prompt x, and let $y_1, \cdots, y_N \overset{\text{i.i.d.}}{\sim} \pi_\theta(\cdot \mid x)$ be our independently sampled N generations and let $f : \mathcal{X} \times \mathcal{Y}^{\oplus n} \to \mathbb{R}$ be our set objective. Then*

$$\mathbb{E}\left[\sum_{i=1}^N \nabla_\theta \log \pi_\theta(y_i \mid x)\, \widehat{A^\sharp_{\text{marg}}}(x, y_i)\right] = \frac{N}{n}\nabla_\theta \mathbb{E}_{y_{1:n} \sim \pi_\theta(\cdot|x)}\big[f(x, y_{1:n})\big].$$

Consequently, after also taking expectation over $x \sim \mathcal{D}$ and scaling the learning rate by $\frac{n}{N}$, the estimator is an unbiased estimator of the set RL gradient.

Note that it is not necessary for us to construct all $K = \binom{N}{n}$ sets of size n. There are several alternatives that one can use and still arrive at an unbiased estimator of eq. (6). One straightforward example is as follows: we first drawn N generations, $y_1, \cdots, y_N \sim \pi_\theta(\cdot \mid x)$. We know that there are, altogether, $\binom{N}{n}$ sets of size n one can construct from these N generations (without replacement). Now, one can uniformly sample K sets from this collection of all $\binom{N}{n}$ sets. This also gives us an unbiased estimator of the set RL gradient, by using an *incomplete* U-statistic.

Proposition 2.6. *Fix a prompt x, and let $y_1, \cdots, y_N \overset{\text{i.i.d.}}{\sim} \pi_\theta(\cdot \mid x)$ be our independently sampled N generations and let $f : \mathcal{X} \times \mathcal{Y}^{\oplus n} \to \mathbb{R}$ be our set objective. Let $G_1, \cdots, G_K$ be our K sets of size n and, suppose, these are sampled uniformly, i.e.*

$$G_1, \cdots, G_K \sim \text{Uniform}(\{G_j\}_{j=1}^{\binom{N}{n}}).$$

Then

$$\mathbb{E}\left[\sum_{i=1}^{N} \nabla_\theta \log \pi_\theta(y_i \mid x)\, \widehat{A^\sharp_{\text{marg}}}(x, y_i)\right] = \alpha' \nabla_\theta \mathbb{E}_{y_{1:n} \sim \pi_\theta(\cdot|x)}\big[f(x, y_{1:n})\big]$$

where $\alpha' \in \mathbb{R}$. Consequently, after also taking expectation over $x \sim \mathcal{D}$ and scaling the learning rate, the estimator is an unbiased estimator of the set RL gradient.

This is potentially useful for settings where scoring a set itself, i.e. computing $f(x, y_{1:n})$ is expensive.

Finally, we consider another example where we address the following question: can we use the set RL framework where the set size itself is not fixed? Suppose we have an objective that can be defined on sets of variable size. How can we optimize this objective over sets of varying size?

Example 2.7. Suppose, we sample N generations in total and we seek to optimize some objective over sets of size $n_1, n_2, \cdots, n_C \in \mathbb{Z}_{\geq 1}$. Here, we assume that the objective function f could be defined on any of these sets (with some abuse of notation). For example, the objective $f(x, y_{1:n}) = \max_i r(x, y_i)$ does not directly depend on the size of the set insofar as the set has at least one generation. We could generalize the ideas we have presented so far to address this problem setting. First, we construct sets of size n_1 and denote the full collection of sets of size n_1 by $\mathcal{G}(n_1)$. For example, we could compute all possible sets of size n_1 combinatorially from the N generations. Similarly, we get a collection of sets of size n_2 called $\mathcal{G}(n_2)$, and so on. Then, we can optimize using set RL by optimizing the following:

$$\mathbb{E}_{y_1, \cdots, y_N \sim \pi_\theta(\cdot|x)}\left[\frac{1}{\binom{N}{n_1}} \sum_{Y \in \mathcal{G}(n_1)} f(x, Y) + \cdots + \frac{1}{\binom{N}{n_C}} \sum_{Y \in \mathcal{G}(n_C)} f(x, Y)\right]$$

3 Analysis

In this section, we analyse the learning dynamics of set reinforcement learning. In particular, we are interested in understanding the characteristics of generations that are encouraged versus discouraged in the set RL framework. We start by recalling the following useful tool:

Definition 3.1 (Scaffold Value)**.** *The scaffold value of a set of generations, $y_{1:n}$, under a policy π and a set-RL objective $f : \mathcal{X} \times \mathcal{Y}^n \to \mathbb{R}$ is defined to be*

$$\Lambda_f(y_{1:n}; \pi, x) := \text{Cov}_{y'_{1:n} \sim \pi(\cdot|x)}\left(f(x, y'_{1:n}), \frac{1}{I(a_{1:n})} \sum_{i,j=1}^{n} 1\{a'_i = a_j\}\right) \tag{11}$$

where $I(a_{1:n})$ is the maximum size of the intersection of $y_{1:n}$ with any other set $y'_{1:n}$. We refer to the space $\Lambda_f(\pi) = \{(a_{1:n}, \Lambda_f(a_{1:n}; \pi) \mid a_{1:n} \in \mathcal{A}^n\}$ as the scaffold of the policy π.

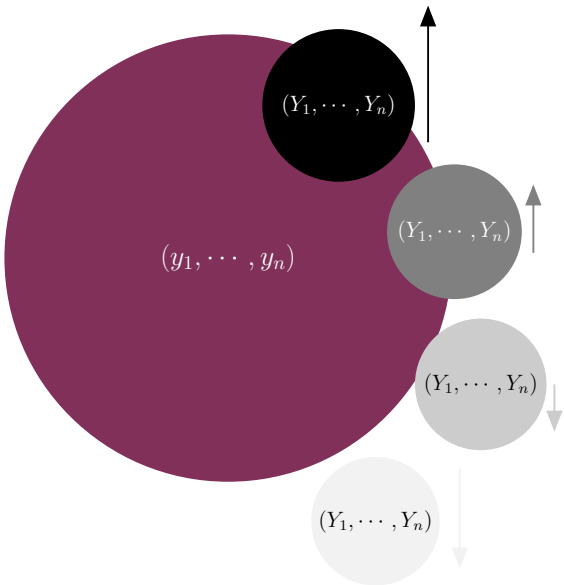

Figure 2: Visualization of a scaffold value of a fixed set of generations, $(y_1, \cdots, y_n)$. Here $(Y_1, \cdots, Y_n) \sim \pi_\theta(\cdot \mid x)$ are sets of generations being sampled and the arrows indicate the value of $f(x, Y_{1:n})$. In this visualization, larger intersection between $y_{1:n}$ and the sampled $Y_{1:n}$ tends to correlate with higher scores under the objective f. As such, the scaffold value of $y_{1:n}$ is positive.

In particular, when the set is homogeneous, i.e. $y_{1:n} = (y, \cdots, y) =: y^{\oplus n}$*, the scaffold value can be, equivalently, written as:*

$$\Lambda_f(y^{\oplus n}; \pi, x) = \pi_\theta(y \mid x) \left(\mathbb{E}_{y_{2:n} \sim \pi_\theta(\cdot|x)} \left[f(x, y, y_{2:n})\right] - \mathbb{E}_{y_{1:n} \sim \pi_\theta(\cdot|x)} \left[f(x, y_{1:n})\right]\right).$$

We also look at the closely related marginal set advantage of a generation:

Definition 3.2 (Marginal Set Advantage)**.** *The marginal set advantage of a generation* $y \in \mathcal{Y}$ *under a policy* π *and a set-RL objective* f *is defined as*

$$A^\sharp_{\text{marg}}(x, y; \pi) = \mathbb{E}_{y_{2:n} \sim \pi_\theta(\cdot|x)} \left[f(x, y, y_{2:n})\right] - \mathbb{E}_{y_{1:n} \sim \pi_\theta(\cdot|x)} \left[f(x, y_{1:n})\right]. \tag{12}$$

Note that $A^\sharp_{\text{marg}}(x, y; \pi) = \frac{\Lambda_f(y^{\oplus n}; \pi, x)}{\pi(y|x)}$.

The marginal advantage directly determines which generations' log likelihood increases in a set RL algorithm. Suppose, our policy is parametrized as a softmax distribution:

$$\pi_\theta(y \mid x) = \frac{\exp\left(z_{xy}\right)}{\sum_{y'} \exp\left(z_{xy'}\right)}$$

where z_{xy} is the output logit of y from x. Then, we have the following result determining how the logit changes for a specific generation.

Lemma 3.3 (Logit shift in set RL)**.** *In set reinforcement learning, assuming the objective function* f *is symmetric in its arguments and the learning rate is* α*, the shift in logit after one step gradient update from* π_θ^k *to* π_θ^{k+1} *can be written as:*

$$z_{xy}^{k+1} - z_{xy}^k = \alpha \pi_{\theta^k}(y \mid x) \left[\mathbb{E}_{y_{2:n} \sim \pi_\theta^k(\cdot|x)} \left[f(x, y, y_{2:n})\right] - \mathbb{E}_{y_{1:n} \sim \pi_\theta^k(\cdot|x)} \left[f(x, y_{1:n})\right]\right]. \tag{13}$$

We now compute the marginal set advantages under some objectives to better understand their learning dynamics.

Example 3.4. Let $f(x, y_{1:n}) = \max_{i \in \{1,\cdots,n\}} r(x, y_i)$. Suppose that the probability of sampling a successful generation is p. Then, one can easily show that for $f(x, y_{1:n}) = \max_{i \in \{1,\cdots,n\}} r(x, y_i)$ and for a fixed *incorrect generation* y (i.e. $r(x, y) = 0$),

$$\begin{aligned} A^{\sharp}_{\text{marg}}(x, y) &= \mathbb{E}_{y_{2:n} \sim \pi_\theta(\cdot|x)} \left[f(x, y, y_{2:n}) \right] - \mathbb{E}_{y_{1:n} \sim \pi_\theta(\cdot|x)} \left[f(x, y_{1:n}) \right] \\ &= (1 - (1-p)^{n-1}) - (1 - (1-p)^n) \\ &= (1-p)^n - (1-p)^{n-1}, \end{aligned}$$

which is non-positive for any $p \in [0, 1]$. As such, under this objective, incorrect generations' log likelihood is never increased by our set RL algorithm.

Furthermore, the notion of scaffold values helps us understand how entropy changes during set RL gradient updates as well.

Proposition 3.5. *Consider the set RL setup on $x \in \mathcal{X}$. After one update to the policy, the change in entropy, $\Delta = \mathcal{H}\left(\pi_\theta^{k+1} \mid x\right) - \mathcal{H}\left(\pi_\theta^{k} \mid x\right)$, is given by*

$$\Delta \approx -\alpha \text{Cov}_{y_{1:n}} \left(\frac{1}{n} \sum_{i=1}^{n} \log \pi_\theta^k(y_i \mid x), \text{Cov}_{y'_{1:n}} \left(f(x, y'_{1:n}), \sum_{i,j=1}^{n} 1\{y_i = y'_j\} \right) \right),$$

where both covariances are taken with respect to $\pi_\theta^k(\cdot \mid s)$ and α is the learning rate.

4 Polychromic Objectives

Exploration is a critical component in learning from experience. Firstly, it is necessary if you want to train an agent in a complex problem setting where one must explore to discover at least one successful solution; note that if the agent fails to discover any behaviour that gets some reward signal, then policy gradient methods will simply not be able to optimize at all since the advantage of all actions will be 0. Secondly, often it is quite important for the agent to discover several, diverse successful behaviours. An agent that is learning to play chess should ideally master multiple opening lines rather than relying on a single strategy that may prove ineffective against a different opponent at test time. Thirdly, exploration is necessary for an agent to stress test what it has learned and its internal beliefs. Otherwise, the model overfit on spurious correlations between the reward signal and specific behaviours.

The exploration-exploitation tradeoff in reinforcement learning is a well-known problem. This problem becomes especially pronounced in policy gradient methods. One must train the agent to learn what gets high rewards (i.e. exploit) while also attempting novel strategies (i.e. explore). This requires a form of *optimism under uncertainty*: the agent must attempt strategies during data collection that are exploratory even if they do not immediately get high rewards. Since the data is sampled from the policy itself, the learning signal itself must encourage allocating probability mass to strategies that have not yet demonstrated high reward. For example, when attempting to solve an algorithmic problem, an agent exploring a dynamic programming formulation should continue refining that strategy even if its initial implementations fail unit tests, rather than immediately reverting to a familiar but less scalable exhaustive search approach.

However, the learning signal cannot simply encourage attempting novel strategies. It must also encourage the policy to exploit by re-attempting what has already receive high rewards. As such, the algorithm must address the *trade-off* between exploration and exploitation. We seek an optimization objective for reinforcement learning that satisfies the following desiderata. First, the objective should encourage *optimistic exploration* by assigning positive learning signal to trajectories that attempt novel reasoning strategies, even when those strategies have not yet yielded high reward. Second, the objective should *intrinsically encode the exploration–exploitation trade-off* and incentivize balancing these two goals. This allows the model itself to learn how to balance these competing objectives through optimization.

Polychromic objectives are a family of objectives that are designed to do that. We first look at a practical example of a polychromic objective in the same framework as section 2.2:

$$f_{\text{poly}}(x, y_1, \cdots, y_n) = \frac{1}{n} \sum_{i=1}^{n} r(x, y_i) \cdot d(x, y_1, \cdots, y_n), \tag{14}$$

where $d(x, y_1, \cdots, y_n)$ is a function that measures the diversity of the set of generations, $y_{1:n}$. Here, we assume that $r(x, y) \in [0, 1]$ and $d(x, y_{1:n}) \in [0, 1]$. Because the set-RL gradient uses a shared advantage for all trajectories in a set, this objective increases the likelihood of successful behaviours *and* diverse exploratory trajectories. The shared advantage term amplifies exploratory trajectories that do not (yet) yield high rewards, pushing the policy to discover diverse strategies. We formalize this analysis now.

First, we analyse the marginal set advantage of a generation (see eq. (12)) under the polychromic objective. Fix a generation $y \in \mathcal{Y}$ and assume that $\pi_\theta(y \mid x) > 0$. Recall that in set reinforcement learning, a generation y receives increased probability mass (i.e., $\log \pi_\theta^{k+1}(y \mid x) - \log \pi_\theta^{k}(y \mid x) > 0$) if and only if it has a positive marginal set advantage and non-zero probability under the rollout policy. The marginal set advantage of y can be decomposed as follows:

$$
\begin{aligned}
& A^{\sharp}_{\text{marg}}(x, y; f_{\text{poly}}, \theta^k) \\
= \; & \underbrace{(\frac{1}{n} r(x, y) + \frac{n-1}{n} \mathbb{E}_Y[r(x, Y)]) \mathbb{E}_{Y_{2:n}}[d(x, y, Y_{2:n})]}_{\text{Term 1: Mean reward of sets containing } y \times \text{ Mean diversity of sets containing } y} - \mathbb{E}_Y[r(x, Y)]\, \mathbb{E}_{Y_{1:n}}[d(x, Y_{1:n})] \\
& + \underbrace{\text{Cov}_{Y_{2:n}}(\frac{1}{n} r(x, y) + \frac{1}{n} \sum_{i=2}^{n} r(x, Y_i), d(x, y, Y_{2:n}))}_{\text{Term 2: Exploration–Exploitation alignment in sets containing } y} - \text{Cov}_{Y_{1:n}}(r(x, Y_1), d(x, Y_{1:n}))\,. \qquad (15)
\end{aligned}
$$

We now interpret this advantage:

- *Term 1* reflects the expected contribution of generation y to the product of reward and diversity in sets that contain it. Importantly, this term depends on both the reward and diversity contributions of other generations in the set. Consequently, even if y itself does not contribute reward (or diversity), this term can still be positive provided other sampled generations contribute reward (or diversity) in expectation.

- *Term 2* captures how the presence of generation y affects the covariance between reward and diversity across sets containing y. Intuitively, this term favors generations that enable sets to simultaneously achieve high reward and high diversity. Thus, generations that help align exploration with exploitation receive positive learning signal.

Therefore, even when $r(x, y) = 0$, a generation can still receive increased probability mass if it contributes diversity while other generations contribute reward, or if it helps produce sets in which reward and diversity are positively aligned. This directly reflects the two desiderata that motivate our method. First, the objective supports *optimistic exploration*: trajectories that attempt novel reasoning strategies can receive positive learning signal even before they become individually successful. Second, the objective *intrinsically encodes the exploration–exploitation trade-off*: the advantage assigned to a trajectory depends in part on whether it helps the model form sets that jointly achieve high reward and high diversity, rather than on a manually tuned weighting coefficient. In this sense, the policy update itself internalizes the balance between exploration and exploitation.

This has a few features.

- In set reinforcement learning with polychromic objectives, as a model's accuracy improves, exploratory generations can still receive positive learning signal even when incorrect. Because the set objective shares credit among all generations in a set, incorrect but diverse generations can benefit from the presence of correct generations in the same set. Consequently, the incentive for exploration does not vanish as success rate improves.

- Furthermore, the algorithm naturally assigns advantage based on how a generation contributes to aligning exploration and exploitation across sets. This behavior arises from two components: the set RL framework and the multiplicative structure of the polychromic objective - both of which are necessary for the covariance term to emerge in the marginal set advantage.

Note that this analysis can be extended to any objective of the form $f(x, y_{1:n}) = \varphi(x, y_{1:n})\psi(x, y_{1:n})$. Insofar as these are valid objectives for set reinforcement learning that do not collapse into standard reinforcement learning, the algorithmic framework will encourage generations that satisfy both (even if they do not strictly satisfy the other, as long as other generations can compensate) while also strongly encouraging generations that help the model achieve a positive covariance between φ and ψ. One can imaging objectives of this form where $\varphi(x, y_{1:n}) = \frac{1}{n}\sum_{i=1}^{n} r_A(x, y_i)$ and $\psi(x, y_{1:n}) = \frac{1}{n}\sum_{i=1}^{n} r_B(x, y_i)$, where r_A and r_B are two different reward functions that we want our model to satisfy in tandem.

We now come to the generalized definition of the polychromic objective. The motivation for the polychromic objective comes from understanding the entropy change in set reinforcement learning as shown in Proposition 3.5. We see that entropy collapses when there is a positive covariance between the log probabilities of generations in a set and the scaffold value of the generations. Note that if this covariance is always positive, we would get entropy collapse. The idea of a polychromic objective is to ensure that the entropy is never allowed to collapse on a set of generations that is a singleton. In other words, the policy can never collapse onto sampling a single generation.

Definition 4.1. *A polychromic objective is a function*

$$\varphi : \mathcal{X} \times \mathcal{Y}^n \to \mathbb{R}$$

that factors as

$$\varphi(x, y_{1:n}) = \varphi^{(R)}(x, y_{1:n})\varphi^{(d)}(x, y_{1:n}),$$

where $\varphi^{(R)}$ and $\varphi^{(d)}$ are scalar-valued functions satisfying:

1. $\mathrm{Cov}_{y_{1:n}\sim\pi_\theta(\cdot|x)}(\varphi^{(R)}(x, y_{1:n}), \sum_{i=1}^{n} r(x, y_i)) > 0$,
2. $\mathrm{Cov}_{y_{1:n}\sim\pi_\theta(\cdot|x)}\left(\varphi^{(d)}(x, y_{1:n}), \sum_{i=1}^{n} 1\{y_i = y\}\right) < 0$ *for any y, and*
3. $\varphi^{(R)}(x, \cdot)$ *and* $\varphi^{(d)}(x, \cdot)$ *share the same range, i.e.,*

$$\inf \varphi^{(R)}(x, \cdot) = \inf \varphi^{(d)}(x, \cdot), \quad \sup \varphi^{(R)}(x, \cdot) = \sup \varphi^{(d)}(x, \cdot),$$

where the infimum and supremum are taken over all sets $y_{1:n}$.

The practical polychromic objective we have seen before aims to ensure the second requirement using a diversity function since it is, often, intractable to find an objective that satisfies the requirement for all $y \in \mathcal{Y}$.

We confirm now, through a preliminary analysis, that polychromic objectives indeed satisfy the motivation behind it.

Proposition 4.2. *For any homogeneous set $y_{1:n} = \{y\}$ where $r(x, y) = 1$, there exists $\epsilon \in (0, 1)$ such that $\Lambda_{f_{\text{poly}}}(y^{\oplus n}; \pi, x) < 0$ when $\pi_\theta(y \mid x) > \epsilon$. Furthermore, the scaffold values of these homogeneous sets satisfy the bound $\Lambda_{f_{\text{poly}}}(y; \pi, x) \leq \sqrt{\frac{p(1-p)}{n}}$.*

This result shows that once a successful generation y accumulates sufficient probability mass, the polychromic objective automatically prevents further entropy collapse onto sets that only contain this action and, when we use larger sets in the set RL framework, the upper bound on the scaffold value of a homogeneous set decreases. This is desirable since it suggests that our policy learns to generate this action without incessantly collapsing probability mass on such homogeneous sets.

Proposition 4.3. *Suppose $y_{1:n}$ is heterogeneous where each y_i is unique with probability $p \in (0, \frac{1}{n})$. Suppose exactly q of the n actions satisfy $r(x, y_i) = 1$, and that any other action $y' \notin y_{1:n}$ with $\pi_\theta(y' \mid x) > 0$ yields $r(x, y') = 0$. Then, the scaffold value of $y_{1:n}$ satisfies $\Lambda_{f_{\text{poly}}}(y_{1:n}; \pi, x) > \frac{qp^n(1-p)}{n}$.*

Note that the set in this proposition includes unsuccessful generations as well that contribute diversity. As such, there are, likely, several such heterogeneous sets (provided that the size of sets n is large enough) with positive scaffold values that attract more probability mass than homogeneous sets. Furthermore, the lower bound guarantee increases as the number of successful actions in the set increases. The polychromic objective therefore channels entropy collapse toward those sets that balance success and exploration, rather than permitting collapse onto homogeneous behaviors.

5 Algorithms for Set RL

In this section, we present some examples of algorithms in set reinforcement learning.

5.1 Poly-EPO

Polychromic Exploratory Policy Optimization (Poly-EPO) is an algorithm that leverages the general recipe in algorithm 1 for optimizing the practical polychromic objective.

We first discuss the implementation of the diversity function. There are a few considerations to keep in mind. Firstly, we require the computation of the diversity of each set, i.e. $d(x, y_1, \cdots, y_n)$, to be computationally practical. This is because our set RL recipe constructs all $K = \binom{N}{n}$ sets of size n from the N generations per prompt; if scoring each set under the polychromic objective is computationally expensive, then the runtime complexity of our algorithm will increase rapidly as we scale up the algorithm to a setting where we sample more generations per prompt. Secondly, we much be able to able to do this computation for a wide variety of problem settings. Language models are post-trained on problems in mathematics, coding, instruction-following, etc., and the training set for each of these settings can be quite large. We require a method for measuring diversity of sets of generations that can be easily transferred to a wide variety of settings.

The main ingredient we use is an language model judge (LM-judge) that is used to cluster the responses sampled from our policy. Since LM-judges are already incorporated in post-training methods in a wide variety of settings such as proof-writing and non-verifiable domains, this allows Poly-EPO to benefit from the post-training infrastructure that is already widely adopted. Furthermore, clustering responses based on semantic similarity is a substantially simpler task than solving the problem, which allows us to leverage the generator-verifier gap in designing our method.

Recall that, for each prompt x, we first sample N independent generations and, then, construct sets of size n from them. We first take all N generations conditioned on the same prompts and ask an LM-judge to cluster them based on their semantic similarity in their reasoning strategy. Let $\mathcal{C}(y_i) \in \{1, \cdots, N\}$ be the cluster to which generation y_i belongs. Then, we define the diversity function for each set $\{y_{(j_1)}, \cdots, y_{(j_n)}\}$ to be:

$$d\big(x, y_{(j_1)}, \ldots, y_{(j_n)}\big) = \frac{\text{No. of clusters in } (y_{(j_1)}, \ldots, y_{(j_n)})}{n} = \frac{\left|\left\{\mathcal{C}\big(y_{(j_1)}\big), \ldots, \mathcal{C}\big(y_{(j_n)}\big)\right\}\right|}{n}$$

Note that $d(x, y_{(j_1)}, \cdots, y_{(j_n)}) \in [0, 1]$, which is required for the polychromic objective. To increase the reliability of the judge, we use in-context learning to steer the judge to cluster them based on the strategy (both high-level and low-level) used in each generation, as opposed to differences in tone, style, etc.

Now, we have all the ingredients for computing the score of each set under the polychromic objective. Since scoring each set is computationally straightforward, we can construct all $K = \binom{N}{n}$ sets in our set RL recipe from algorithm 1.

References

[HOX+26] Jubayer Ibn Hamid, Ifdita Hasan Orney, Ellen Xu, Chelsea Finn, and Dorsa Sadigh. Polychromic objectives for reinforcement learning, 2026.

[KL02] Sham Kakade and John Langford. Approximately optimal approximate reinforcement learning. In *Proceedings of the Nineteenth International Conference on Machine Learning*, ICML '02, page 267–274, San Francisco, CA, USA, 2002. Morgan Kaufmann Publishers Inc.

[WK25] Christian Walder and Deep Karkhanis. Pass@k policy optimization: Solving harder reinforcement learning problems, 2025.